

Structuring LLM Arguments: A Typed Attack Candidate Approach for Dung Abstract Argumentation Frameworks

A. Habyarimana

Johannes Gutenberg-Universität Mainz, Mainz, Germany
`ahabyari@students.uni-mainz.de`

Abstract. This document describes the current state of an LLM-based argument mining pipeline that extracts Dung Abstract Argumentation Frameworks (AAFs) from natural-language debate texts. Rather than treating attack detection as a binary classification task, the system prompts a Large Language Model to produce *typed attack candidates* — each annotated with an attack type (rebuttal, undercutting, or undermining), a numerical confidence score, and a natural-language justification. A two-stage confidence filter selects candidates; accepted candidates are collapsed to binary Dung `att/2` facts and evaluated in Prolog under grounded, preferred, and stable semantics. The document is intended as a technical reference for the supervisors and describes the pipeline in detail, including pseudocode, design decisions, and example outputs from a run on the UKP nuclear energy debate corpus.

Keywords: Argument Mining · Dung AAF · Large Language Models · Attack Typology · UKP Corpus

1 Introduction

The goal of this project is to connect two fields that have largely developed in parallel: *argument mining* (identifying argumentative structure in natural text) and *formal argumentation* (computing acceptable positions from an attack graph).

The core idea is simple: if we can identify which arguments attack which others in a debate, we obtain a Dung Abstract Argumentation Framework [2] for free. A Prolog solver then computes which arguments survive under various semantics — without any further LLM involvement. The formal result can then be compared with a direct “what do you find convincing?” query to the same LLM, revealing whether the LLM’s own intuitions agree with the formal consequences of the attacks it identified.

The key methodological decision in this project is **not** to ask the LLM a binary yes/no question per argument pair. Instead, the LLM proposes a structured *attack candidate* that includes: (1) an attack type drawn from Pollock’s typology [4], (2) a confidence score, and (3) a one-sentence justification. This richer output improves the quality of the extraction and makes the system’s reasoning transparent and auditable.

2 Background

2.1 Dung Abstract Argumentation Frameworks

A Dung AAF [2] is a pair $AF = \langle \mathcal{A}, \mathcal{R} \rangle$ where $\mathcal{A}$ is a finite set of arguments and $\mathcal{R} \subseteq \mathcal{A} \times \mathcal{A}$ is a binary attack relation. The internal structure of arguments is irrelevant — only the graph matters for computing extensions.

A set $S \subseteq \mathcal{A}$ is *conflict-free* if no member attacks another. Argument a is *defended* by S if every attacker of a is attacked by some member of S . Three standard semantics are used in this project:

- **Grounded extension:** the unique least fixed point of $F_{AF}(S) = \{a \in \mathcal{A} \mid S \text{ defends } a\}$, computed by iterating from the empty set. Always exists and is unique.
- **Preferred extensions:** maximal admissible sets (conflict-free and self-defending). May be multiple.
- **Stable extensions:** conflict-free sets that attack every non-member. May not exist.

2.2 Attack Typology

Dung’s framework is *abstract*: it does not distinguish why one argument attacks another. Pollock [4] and later ASPIC+ [3] identify three qualitatively distinct attack types in natural-language debate:

Rebuttal The attacker directly contradicts the *conclusion* of the target. This is the strongest form of attack — it directly denies what the target claims.

pro: “Nuclear energy is a clean energy source.”

con attacks via rebuttal: “Nuclear plants produce catastrophic environmental contamination.”

Undercutting The attacker shows that the target’s conclusion does not follow from its premises, even if those premises are true. The inference step is attacked, not the conclusion directly.

pro: “Newer reactor designs are safer, so nuclear is acceptable.”

pro attacks via undercutting: “Even newer designs show that safety improvements do not generalise — Chernobyl-era reactors were also considered safe in their time.”

Undermining The attacker targets a *premise* the target relies on, without directly engaging the conclusion.

pro: “Nuclear plants have robust containment structures, so they are safe.”

con attacks via undermining: “Nuclear waste is hard to manage and radiation cannot be contained indefinitely.” (attacks the premise that containment is sufficient)

All three types collapse to a single $\text{att}(A, B)$ in Dung’s framework. Section 3.5 discusses this design decision.

Table 1. UKP corpus statistics for *Nuclear Energy*.

Topic	Sentences	Documents	Total Args	Pro / Con
Nuclear Energy	6,261	144	2,469	354 / 742

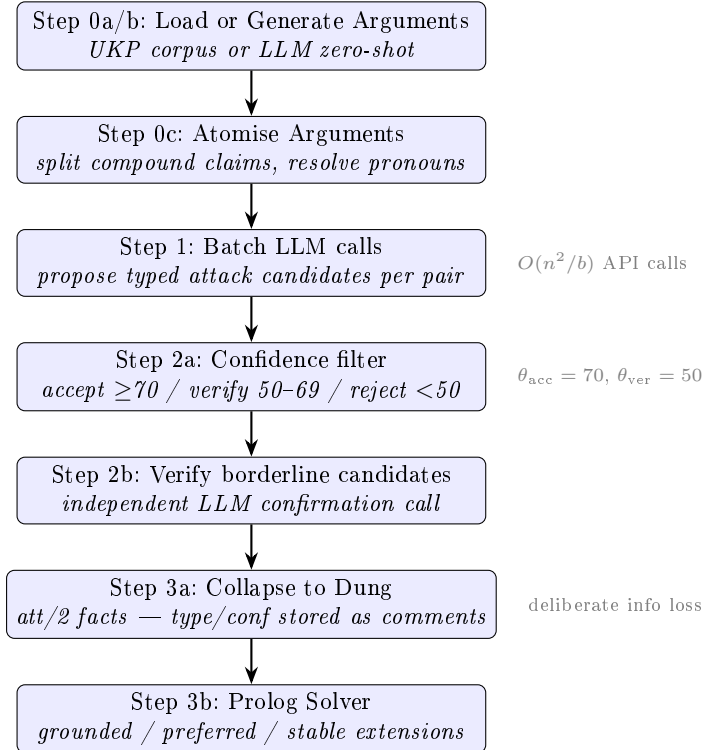
2.3 Dataset: UKP Sentential Argument Mining Corpus

The pipeline is evaluated on the UKP Sentential Argument Mining Corpus [5], which provides sentence-level stance annotations for eight controversial topics. Table 1 shows the corpus statistics for the topic used in initial experiments.

Each sentence is labelled **Argument_for**, **Argument_against**, or **NoArgument**. The pipeline loads only labelled arguments (discarding **NoArgument**) and uses the **test** split.

3 Pipeline

Figure 1 shows the full pipeline. The following subsections describe each stage in detail.

**Fig. 1.** Full pipeline from raw text to formal Dung extensions.

Algorithm 1 Argument Atomisation

Require: argument list $\mathcal{A}$, topic T **Ensure:** atomised list $\mathcal{A}'$

```

1:  $\mathcal{A}' \leftarrow []$ 
2: for each  $a \in \mathcal{A}$  in parallel do
3:    $atoms \leftarrow \text{LLM\_atomise}(a, T)$   $\triangleright$  returns  $\geq 1$  atomic args
4:   append  $atoms$  to  $\mathcal{A}'$ 
5: end for
6: fix ID conflicts in  $\mathcal{A}'$   $\triangleright$  e.g.  $\text{pro5} \rightarrow \text{pro5}, \text{pro5b}$  return  $\mathcal{A}'$ 

```

3.1 Step 0a/b: Argument Acquisition

UKP source. Arguments are loaded from the corpus TSV files. Only rows with annotation `Argument_for` or `Argument_against` are kept. A balanced sample (equal pro/con) is drawn with a fixed random seed for reproducibility. Each argument is represented as a Python dictionary:

```

{"id": "pro5", "text": "Nuclear power can provide ...", "
  stance": "pro"}

```

LLM source. Alternatively, arguments are generated by prompting the LLM to produce n_{pro} pro and n_{con} con arguments about a given topic in the same JSON format.

3.2 Step 0c: Argument Atomisation

Natural-language arguments frequently bundle multiple claims. A compound sentence like “Although nuclear energy produces waste, it is cleaner than coal” contains a concession and a main claim — the concession is a rhetorical hedge, not a standalone argument.

The atomisation step uses an LLM to split each argument into atomic claims — those that express exactly one evaluative position. The prompt enforces three rules:

1. **Drop concessive fragments.** Anything introduced by “Although”, “While”, “Despite”, etc. is dropped.
2. **Resolve pronouns.** Any fragment containing “it”, “they”, “this” without a recoverable referent is either resolved or dropped.
3. **Filter non-argumentative content.** Pure factual statements without an evaluative stance are removed.

Algorithm 1 shows the procedure. In practice, roughly half of UKP sentences are split into 2 atomic arguments.

Algorithm 2 Typed Attack Candidate Extraction**Require:** atomised arguments $\mathcal{A}'$, topic T , batch size b **Ensure:** candidate dict C : $(a_i, a_j) \rightarrow \text{candidate}$

```

1:  $\text{pairs} \leftarrow \{(a_i, a_j) \mid i < j, \text{base}(a_i) \neq \text{base}(a_j)\}$ 
2:  $C \leftarrow \emptyset$ 
3:  $\text{batches} \leftarrow \text{chunk}(\text{pairs}, b)$ 
4: for each  $\text{batch} \in \text{batches}$  in parallel do
5:    $\text{raw} \leftarrow \text{LLM\_batch}(\text{batch}, T)$ 
6:   if valid JSON list then
7:     insert results into  $C$ 
8:   else
9:     retry batch; then fall back to individual calls
10:  end if
11: end for return  $C$ 

```

3.3 Step 1: Typed Attack Candidate Extraction

This is the central contribution of the pipeline. Instead of asking the LLM “does A attack B ?” (binary), the prompt asks it to *propose a typed attack candidate* with the following fields:

```

{
  "pair":          "pro55:con72",
  "direction":     "A_ATTACKS_B | B_ATTACKS_A | NONE",
  "attack_type":   "rebuttal | undercutting | undermining |
                    NONE",
  "confidence":    "<integer 0--100>",
  "reason":        "<one-sentence justification>"
}

```

The prompt provides the full definitions of all three attack types (Section 2.2) and enforces: *if $\text{direction}=\text{NONE}$ then $\text{attack_type}=\text{NONE}$ and $\text{confidence}=0$* . Mutual attacks (MUTUAL) are not a valid direction value; if both directions hold, the LLM is expected to return two separate candidates — one per direction — in subsequent pair orderings (see Section 5).

Batching. All $\binom{|\mathcal{A}'|}{2}$ argument pairs are grouped into batches of size $b = 10$ and submitted as a single API call each. This reduces the number of API calls from $O(n^2)$ to $O(n^2/b)$. Pairs for which the LLM returns malformed JSON are retried once as individual calls.

Algorithm 2 shows the extraction procedure.

Why typed candidates? The type annotation guides the LLM’s own reasoning: by being asked to *justify which type applies*, the model is forced to engage with the argumentative content rather than pattern-matching on surface cues such as opposite stances. The reason field provides an auditable trace for each decision.

Table 2. Example filter outcomes from the nuclear energy run (10 UKP arguments, 30 atoms after atomisation).

Pair	Direction	Conf	Type	Outcome
pro55 $\leftrightarrow$ con72	B_ATTACKS_A	80	undermining	ACCEPT
pro106 $\leftrightarrow$ con42	B_ATTACKS_A	95	rebuttal	ACCEPT
pro62b $\leftrightarrow$ con126	B_ATTACKS_A	90	rebuttal	ACCEPT
pro55b $\leftrightarrow$ con72	B_ATTACKS_A	65	undermining	REJECT (verify denied)
pro27 $\leftrightarrow$ con72	B_ATTACKS_A	60	undermining	REJECT (verify denied)
pro62 $\leftrightarrow$ pro106	A_ATTACKS_B	60	undermining	REJECT (verify denied)
pro5b $\leftrightarrow$ con72	NONE	10	NONE	SKIP

3.4 Step 2: Confidence Filtering and Verification

Candidates are routed through a three-way decision based on their reported confidence c :

$$\text{decision}(c) = \begin{cases} \text{ACCEPT} & c \geq \theta_{\text{acc}} = 70 \\ \text{VERIFY} & \theta_{\text{ver}} \leq c < \theta_{\text{acc}}, \quad \theta_{\text{ver}} = 50 \\ \text{REJECT} & c < \theta_{\text{ver}} \end{cases} \quad (1)$$

Candidates in the **VERIFY** region are submitted to an independent LLM call (the *verifier*) that answers: “Does this attack relation genuinely hold?” and returns `{"confirms": true/false, "reason": "..."}.` A verified candidate is accepted; an unconfirmed one is rejected.

Table 2 illustrates the three outcomes with real examples from the nuclear energy run.

3.5 Step 3a: Collapsing to Dung

Accepted candidates — each a rich record $\langle A, B, \text{type}, c, r \rangle$ — are reduced to binary Dung facts:

$$\langle A, B, \text{type}, c, r \rangle \mapsto \text{att}(A, B). \quad (2)$$

This is a **deliberate design decision**. Three alternatives were considered:

1. *Collapse (chosen)*: All accepted candidates become **att/2** facts regardless of type. Type/confidence are preserved as Prolog comments.
2. *Extended framework*: Use ASPIC+ or Bipolar AF to preserve type information in the formal semantics. More expressive, but substantially more complex to implement correctly.
3. *Type-conditioned filtering*: Use separate confidence thresholds per attack type before collapsing (e.g., require confidence ≥ 80 for undermining, ≥ 70 for rebuttal).

We chose option (1) because: (a) Dung semantics are well-understood and the implementation is verified; (b) the type information is *not lost* — it is retained as metadata for analysis; (c) the primary research question is whether typed prompting *improves extraction quality* relative to binary prompting, independently of the formal evaluation.

The resulting Prolog file looks like this:

```
% arg/1 facts
arg(pro55). % [pro] Nuclear plants have substantial
            containment structures.
arg(con72). % [con] Nuclear energy results in catastrophic
            env. contamination.

% att/2 facts (Dung 1995 -- binary; type/conf from LLM
              candidates as comment)
att(con72, pro55). % [undermining] conf=80 -- "
                  Catastrophic contamination
                  % attacks the premise that
                  containment ensures safety."
att(con42, pro106). % [rebuttal] conf=95 -- "Uranium is
                  limited, directly
                  % contradicting a persistent supply
                  claim."
```

3.6 Step 3b: Prolog Solver

The Prolog solver implements all standard Dung semantics [1]:

- **Grounded:** fixed-point iteration of F_{AF} starting from $\emptyset$. Polynomial, always terminates.
- **Preferred:** enumeration of maximal admissible sets via conflict-free power-set generation. Exponential worst-case; restricted to $|\mathcal{A}'| \leq 30$ in the current implementation.
- **Stable:** conflict-free sets that attack all non-members. May not exist.

The solver is parametrised by the AF source file via a Prolog flag and returns results as JSON, which the Python driver reads back.

4 Example Run: Nuclear Energy

We ran the full pipeline on the UKP nuclear energy test split with a balanced sample of 10 arguments (5 pro, 5 con). After atomisation, 30 atomic arguments remained. The $\binom{30}{2} = 435$ pairs (minus same-base-ID pairs: 425) were processed in 43 batches.

Table 3. Attack type distribution — nuclear energy run (30 atoms, 425 evaluated pairs).

Attack Type	Accepted	Share
undermining	–	≈70%
rebuttal	–	≈25%
undercutting	–	≈5%
Total	81	100%

Attack statistics. The pipeline accepted 81 attack relations out of 425 pairs (19%). The type distribution is shown in Table 3.

Note: exact per-type counts to be filled in from full run logs. The distribution here is estimated from the log output shown above.

Formal result. The Prolog solver returned the following extensions:

```

Grounded   : [con119, con126, con168, con168b, con53,
               con72,  con72b, pro2,   pro55b, pro5b, pro62]
Preferred  : [[con119, con126, con168, con168b, con53,
               con72,  con72b, pro2,   pro55b, pro5b, pro62
               ]]
Stable     : [[con119, con126, con168, con168b, con53,
               con72,  con72b, pro2,   pro55b, pro5b, pro62
               ]]

```

The unique, identical grounded, preferred, and stable extension indicates a *well-founded* framework: the attack graph has a clear winner set with no ambiguity. Of the 11 accepted arguments, 7 are con-stance and 4 are pro-stance, suggesting the con side is better defended in this sample.

Selected accepted arguments from the grounded extension:

- **con72:** “Nuclear energy results in catastrophic environmental contamination” (rebuttal against pro55)
- **con168:** “Radiation and nuclear waste are difficult to manage” (undermining against multiple pro arguments)
- **pro62:** “We can make use of solar, wind and other renewable sources of energy” (accepted despite con stance of the debate — survives because it is not attacked)
- **pro2:** “Reactors in the Soviet Union were dangerous, but have improved” (survives; attacks on it are themselves attacked)

5 Design Decisions and Open Questions

No cross-stance filter. An earlier version of the pipeline filtered pairs so that only arguments with opposite stances were checked for attacks. This was removed

because UKP stance labels (*pro* / *con*) reflect the argument’s position in the original text, not a global debate direction. Same-stance arguments can conflict (e.g., two pro-nuclear arguments that prioritise incompatible safety strategies).

Confidence calibration. LLM confidence scores are known to be poorly calibrated. We treat them as routing signals (accept / verify / reject) rather than probabilities. The verification step partially compensates by providing an independent reading for borderline candidates. A future experiment should measure how often the verifier agrees with the classifier as a function of the confidence score.

Batch size vs. fallback rate. With batch size $b = 10$ and 425 pairs, 43 batch calls are made. However, JSON parse failures trigger retries, and individual fallback calls for unresolved pairs add overhead. Observed fallback rate: approximately 20 individual fallback calls per run ($\approx 5\%$).

Scalability. The preferred-extension solver runs in exponential worst-case time and is restricted to $|\mathcal{A}'| \leq 30$. For larger debates, only the grounded extension (polynomial) is computed. This is a hard constraint that should be addressed before scaling to full UKP topics.

References

1. Cerutti, F., Gaggl, S.A., Thimm, M., Wallner, J.P.: Foundations of implementations for formal argumentation. In: Handbook of Formal Argumentation, pp. 319–383. College Publications (2014)
2. Dung, P.M.: On the acceptability of arguments and its fundamental role in non-monotonic reasoning, logic programming and n-person games. Artificial Intelligence **77**(2), 321–357 (1995)
3. Modgil, S., Prakken, H.: The ASPIC⁺ framework for structured argumentation: A tutorial. Argument & Computation **5**(1), 31–62 (2014)
4. Pollock, J.L.: How to reason defeasibly. Artificial Intelligence **57**(1), 1–42 (1992)
5. Stab, C., Gurevych, I.: Identifying argumentative discourse structures in persuasive essays. In: Proceedings of EMNLP 2014. pp. 46–56 (2014)